Use of AI assistants. Problems 9.1–9.4 are No AI. In 9.5, only part (a) is AI allowed. Write code and explanations in the Haskell answer template, renamed after you. Submit `assistant.hs` and, if you use an assistant, the complete `transcript.md` for 9.5. Each problem is worth five points, 25 in all.

Use GHC 9.10.3. Definitions must cover every constructor and list shape. Do not use partial functions such as `head`, `tail`, `!!`, or `fromJust`. Unfinished definitions may remain `undefined` so the file loads. Examples with `undefined` test which computations are demanded.

Problem 9.1. No AI Build a pair of reusable list functions.

(a) Implement `splitAt` by explicit recursion: split a list into its first `n` elements and the remainder, using an empty prefix when `n <= 0`. Use one recursive traversal.

```
splitAt :: Int -> [a] -> ([a], [a])
```

```
ghci> splitAt 3 "abcdef"
("abc","def")
```

(b) Using your `splitAt`, implement `chunksOf`: consecutive chunks of size `n`, with a possibly shorter final chunk. For `n <= 0`, return `[]`. Both functions should work on infinite lists.

```
chunksOf :: Int -> [a] -> [[a]]
```

```
ghci> chunksOf 3 "abcdefgh"
["abc","def","gh"]
ghci> take 2 (chunksOf 3 [1..])
[[1,2,3],[4,5,6]]
```

(c) Explain why only the second expression needs the undefined tail.

```
take 1 (fst (splitAt 3 (1 : undefined)))
length (fst (splitAt 3 (1 : undefined)))
```

Problem 9.2. No AI A search produces lazy `(page, score)` pairs. Higher scores are better; the search should not inspect the page values.

(a) Implement `bestSoFar`. Report the best pair after each candidate, keeping the earlier candidate on a tie. Include unchanged results.

```
bestSoFar :: [(a, Int)] -> [(a, Int)]
```

```
ghci> bestSoFar [("guide",40),("faq",20),("api",80),("index",80)]
[("guide",40),("guide",40),("api",80),("api",80)]
```

(b) Use this stream to implement `search budget target candidates`. Stop at the first score at least `target`, or after at most `budget` candidates, returning the best pair found. Return `Nothing` for empty input or a nonpositive budget. The input may be infinite.

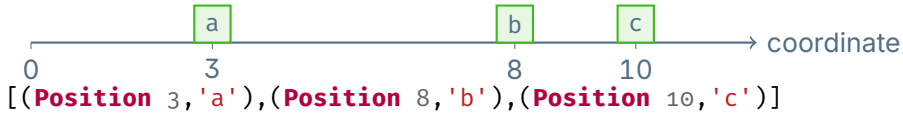
```
search :: Int -> Int -> [(a, Int)] -> Maybe (a, Int)
```

```
ghci> search 2 70 [("guide",40),("faq",20),("api",80)]
Just ("guide",40)
ghci> search 10 70 [("guide",40),("api",80),undefined]
Just ("api",80)
```

(c) For candidates from `map (\page-> (page, score page)) pages`, explain how `search` limits calls to an expensive `score`. Suppose `bestSoFar` returned the first pair and then only pairs with higher scores, skipping unchanged results. Would `search` still respect its budget?

Problem 9.3. No AI In a side-scrolling game, the world may extend indefinitely to the right. Obstacles have positions and labels. Distances represent gaps, shifts, and viewport widths.

```
newtype Distance = Distance Integer deriving (Eq, Show)
newtype Position = Position Integer deriving (Eq, Show)
```



(a) Implement `positions`: convert gaps between consecutive obstacles into absolute positions, starting from coordinate zero. Assume every gap is positive.

```
positions :: [(Distance, a)] -> [(Position, a)]
```

```
ghci> positions [(Distance 3,'a'),(Distance 5,'b'),(Distance 2,'c')]
[(Position 3,'a'),(Position 8,'b'),(Position 10,'c')]
```

(b) Implement `shift` to subtract a distance from each position, and `visible` to keep positions in $[0, width]$ for a nonnegative width. Input positions are strictly increasing; the result must be finite even for an infinite world.

```
shift :: Distance -> [(Position, a)] -> [(Position, a)]
visible :: Distance -> [(Position, a)] -> [(Position, a)]
```

```
ghci> let world = positions [(Distance 3,n) | n <- [1..]]
ghci> visible (Distance 5) (shift (Distance 4) world)
[(Position 2,2),(Position 5,3)]
```

(c) Explain why the displayed pipeline terminates. Why would filtering all obstacles by viewport membership fail to produce a complete result?

(d) Implement `frames width step world` using your earlier functions. The first frame shows the initial viewport; each next frame moves it to the right by `step`. Assume a positive step and a nonnegative width. Return an infinite list of frames, including empty frames.

```
frames :: Distance -> Distance -> [(Position, a)]
       -> [[(Position, a)]]
```

For the `world` from (b), the visible labels are:

```
ghci> take 3 (map (map snd) (frames (Distance 5) (Distance 4) world))
[[1],[2,3],[3,4]]
```

(e) Implement `overlay front back`, merging two worlds in increasing position order. Each input has strictly increasing positions. At a shared position, keep the obstacle from `front`. The function must work when both worlds are infinite.

```
overlay :: [(Position, a)] -> [(Position, a)] -> [(Position, a)]
```

```
ghci> let front = [(Position 3,'a'),(Position 8,'b')]
ghci> let back = [(Position 5,'c'),(Position 8,'d')]
ghci> overlay front back
[(Position 3,'a'),(Position 5,'c'),(Position 8,'b')]
```

Problem 9.4. No AI Use this expression type:

```haskell
data Expr
  = Lit Integer            -- Integer literal
  | Var String             -- Variable name
  | Add Expr Expr          -- Sum of two expressions
  | IfZero Expr Expr Expr -- Condition, zero branch, nonzero branch
  deriving (Eq, Show)

replace :: String -> Expr -> Expr -> Expr
```

`IfZero c yes no` selects `yes` when the value of `c` is zero, and `no` otherwise. The template provides the implementation of `replace x e t`, which replaces occurrences of `Var x` in `t` with `e`.

(a) Implement `prune :: Expr -> Expr`. It removes an `IfZero` when its condition, after pruning, is a literal: choose `yes` for zero and `no` otherwise. Retain the conditional if its pruned condition is not a literal. Prune all retained subexpressions, including conditions. Keep other constructors unchanged; in particular, do not evaluate `Add`.

```
ghci> prune (IfZero (Lit 0) (Add (Lit 2) (Lit 3)) (Var "x"))
Add (Lit 2) (Lit 3)
ghci> prune (Add (IfZero (Lit 1) (Lit 8) (Lit 9)) (Var "x"))
Add (Lit 9) (Var "x")
```

(b) Explain why this pipeline returns `Add (Lit 2) (Lit 3)` without evaluating `undefined`. Discuss `replace` and `prune` separately.

```haskell
prune (replace "flag" (Lit 0)
  (IfZero (Var "flag") (Add (Lit 2) (Lit 3)) undefined))
```

(c) Give two tests with expected results: one where a conditional has another conditional as its condition, and one where its condition is a variable. Explain why pruning preserves values of finite expressions when all variables have values.

(d) For finite, fully defined inputs, do these two orders always produce the same tree?

```haskell
prune (replace x r e)
replace x r (prune e)
```

If they can differ, give a small counterexample. Otherwise, explain why they agree.

(e) Can applying `prune` a second time remove more conditionals from a finite, fully defined expression? Justify your answer from the definition.

Problem 9.5. Summarise a list of names by their count and the longest name's length. Assume finite lists of finite strings, with all counts and lengths fitting in `Int`. Start with:

```
nameStats :: [String] -> (Int, Int)
nameStats xs =
  (length xs, foldr (\s m -> max (length s) m) 0 xs)
```

(a) AI allowed Ask an assistant to improve this function using the prompt below. Save the complete exchange verbatim as `transcript.md`, and its proposed implementation verbatim as `assistant.hs`. Record the service, model, and date. If you choose not to use an assistant, write your own proposed revision in `assistant.hs` and declare that choice.

> Improve this Haskell function so it traverses the input list once and avoids accumulating unevaluated arithmetic. Keep its type and its results on finite lists of fully defined strings. Explain your strictness choices.

Include the definition above. State whether you used an API or a web interface.

(b) No AI Test the proposal against the original on these inputs:

```
[]
["a","long","xy"]
["",""]
```

Record both versions' results. Add a test of your own and explain its purpose.

(c) No AI Compare both versions on `fst (nameStats [undefined])`. Explain any difference in demand and whether it violates the prompt in (a).

Identify when the proposed code evaluates the additions and `max` calls: can unevaluated operations accumulate as it reads more names? Justify your answer from the code.

If you used an assistant, quote one claim from its strictness explanation and justify or correct it using the code or a GHCi check. You need not find a bug to earn full credit.

(d) No AI Write `nameStatsStrict` with the same type, using `foldl'` and forcing both new numeric fields in each step. Use bang patterns or `seq`. Explain why equal results on fully defined inputs need not mean equal behaviour on partial inputs.